1. PARTE TEÓRICA – Relatório Técnico

Título do PDF: Parte Teórica – Entrega Contínua de uma Plataforma de Pedidos em Microsserviços

Autor: Samuel Estevão Sales Cerqueira **Disciplina:** Entrega de Trabalho – DevOps **Data:** 18 de abril de 2026

1. Conceituação de Arquitetura de Microsserviços e o Papel do DevOps em Ambientes Cloud-Native

A arquitetura de microsserviços consiste na decomposição de uma aplicação em serviços pequenos, independentes e focados em um único domínio de negócio. Cada serviço possui seu próprio ciclo de vida, banco de dados e pode ser desenvolvido, implantado e escalado de forma autônoma.

No contexto da Loja Veloz, essa abordagem resolve os problemas atuais de indisponibilidade durante deploys, dificuldade de escalabilidade em picos de tráfego e baixa rastreabilidade de falhas.

O **DevOps** atua como ponte entre as equipes de desenvolvimento e operações, promovendo automação, colaboração e entrega contínua (CI/CD). Em ambientes cloud-native, o DevOps é responsável por conteinerização, orquestração com Kubernetes, pipelines automatizados, observabilidade e Infraestrutura como Código (IaC), reduzindo drasticamente o tempo de entrega e o risco de falhas.

2. Conteinerização: Docker vs Kubernetes

- **Docker** é a tecnologia de conteinerização que empacota a aplicação e suas dependências em imagens leves, portáteis e imutáveis.
- **Docker Compose** permite orquestrar múltiplos containers localmente com um único comando (docker compose up), ideal para desenvolvimento padronizado.
- **Kubernetes** é o orquestrador de containers em produção: gerencia Deployments, Services, escalabilidade automática (HPA/VPA), self-healing, rolling updates e balanceamento de carga.

Utilizamos **Docker Compose no ambiente local** (rápido e reproduzível) e **Kubernetes em produção** (resiliência e escalabilidade).

3. Fundamentação Teórica

- **Orquestração de containers**: Kubernetes (Pods, Deployments, Services, ConfigMaps, Secrets, HPA).
- **CI/CD em ambientes distribuídos**: Pipeline automatizado (build, testes, scan de segurança, publicação de imagens e deploy).
- **Observabilidade (três pilares)**:
  - Métricas (Prometheus + Grafana)
  - Logs (Loki)

    - Traces distribuídos (OpenTelemetry + Jaeger)

O projeto segue os **12-Factor App** (config via variáveis de ambiente, processos stateless, logs como streams, disposability).

4. Justificativa das Principais Decisões Arquiteturais

- Docker Compose + multi-stage builds → ambiente local padronizado e imagens seguras.
- Kubernetes com **Blue-Green Deployment** → zero downtime em campanhas promocionais.
- HPA + VPA → escalabilidade automática baseada em CPU/memória.
- RabbitMQ (mensageria) → comunicação assíncrona via eventos ("PedidoCriado").
- OpenTelemetry → tracing distribuído completo.
- Terraform → reprodutibilidade da infraestrutura (IaC).
- GitHub Actions → CI/CD simples, seguro e gratuito.

Exemplo concreto (obrigatório): Repositório de referência: https://github.com/vhnegrisoli/docker-microservices Projeto brasileiro que demonstra microsserviços de venda e estoque com Docker, RabbitMQ e migração para Kubernetes, exatamente como o desafio da Loja Veloz. Outro excelente case é o artigo "Building a Microservice E-commerce CI/CD Pipeline with GitHub Actions, Docker, and Kubernetes" (Medium, 2025), que implementa pipeline completo para e-commerce.